

1. Resumen general del front-end

El front-end de Sugarbay esta construido con Next.js App Router, React y TypeScript estricto, con un sistema visual retro/vaporwave muy personalizado basado en Tailwind CSS + una capa extensa de estilos en 'app/globals.css'.

La arquitectura mezcla:

- Renderizado de servidor para paginas y carga de datos (catalogos, snapshots, detalle de producto, etc.).
- Componentes cliente para interacciones (header interactivo, modales, buscador global, carritos, filtros, reproductores, carruseles).
- Server Actions y Route Handlers para autenticacion, carrito, checkout y busqueda.

El proyecto no es un front-end generico: tiene un lenguaje visual coherente (Windows 95/98 + neon/vaporwave), reutiliza patrones de tarjetas y ventanas, y combina contenido editorial (conciertos, noticias, media, musica) con e-commerce (store, cart, checkout).

2. Tecnologias utilizadas

Estado actual (implementado)

- Framework principal: 'next@16.2.1' ('package.json').
- UI library: 'react@19.2.4', 'react-dom@19.2.4'.
- Lenguaje: TypeScript con 'strict: true' ('tsconfig.json').
- Estilos:
- Tailwind v4 via '@tailwindcss/postcss' ('postcss.config.mjs').
- Mucha personalizacion en CSS global ('app/globals.css', ~4728 lineas).
- Fuentes:
- Google font 'Press_Start_2P' ('app/layout.tsx').
- Fuentes locales 'ByteBounce.ttf' y 'Retronoid-BZX3.ttf' ('app/layout.tsx' + 'public/fonts').
- Imagenes:
- 'next/image' en la mayoria de componentes.
- Host remoto permitido para ImageKit y YouTube thumbnails ('next.config.ts').
- Validacion:
- 'zod' para auth y checkout ('lib/validators/auth.ts', 'lib/validators/checkout.ts').
- Seguridad/auth:
- Session con cookie + 'jose'/server auth layer.
- Middleware de rutas protegidas en 'proxy.ts'.
- E-commerce/payment:
- Stripe ('app/api/checkout/route.ts', 'app/api/stripe/webhook/route.ts', 'lib/services/stripe.ts').
- ORM y datos:
- Prisma ('@prisma/client') y repositorios 'lib/repositories/*'.
- Recursos externos:
- ImageKit (imagenes), YouTube (embeds y thumbnails), Google Maps (enlaces de venues), Stripe Checkout.

Lo no implementado o no encontrado

- No hay Framer Motion ni libreria de animacion JS externa.

- No hay Redux/Zustand/Context global para estado de UI.
- No hay libreria de formularios tipo React Hook Form (se usa estado local + Server Actions + Zod).

Mejoras posibles

- Incorporar pruebas de UI (Playwright/Cypress) para asegurar regresiones visuales.
- Centralizar mas tokens visuales en variables semanticas para reducir CSS monolitico.

3. Estructura de carpetas y archivos importantes

Estructura principal

- 'app/': rutas y layouts (App Router).
- 'components/': componentes por dominio ('auth', 'cart', 'checkout', 'concerts', 'home', 'layout', 'media', 'music', 'store', 'ui').
- 'lib/': capa de datos, servicios, validadores, utilidades.
- 'public/': assets estaticos y fuentes locales.
- 'prisma/': esquema y seed de datos.

Archivos clave para entender el front-end

- Shell global:
- 'app/layout.tsx'
- 'components/layout/site-header.tsx'
- 'components/layout/header-client.tsx'
- 'components/layout/site-footer.tsx'
- 'components/ui/page-shell.tsx'
- Estetica y sistema visual:
- 'app/globals.css'
- Home:
- 'app/page.tsx'
- 'components/home/home-videos-band.tsx'
- 'components/home/home-header-carousel.tsx'
- Modales e interaccion:
- 'components/ui/app-modal.tsx'
- 'components/layout/global-search.tsx'
- 'components/auth/auth-modal.tsx'
- 'components/cart/cart-drawer.tsx'
- Store/cart/checkout:
- 'app/store/page.tsx'
- 'app/store/[slug]/page.tsx'
- 'app/store/@modal/(.)[slug]/page.tsx'
- 'components/store/store-product-detail-panel.tsx'
- 'components/store/store-product-purchase-form.tsx'
- 'app/carrito/page.tsx'
- 'app/checkout/page.tsx'
- 'components/checkout/checkout-flow.tsx'

Separacion de responsabilidades

- Paginas ('app/**/page.tsx'): seleccionan datos y componen secciones.
- Repositorios ('lib/repositories/**'): lectura/transformacion de datos desde Prisma.
- Componentes visuales ('components/**'): presentacion e interaccion.
- Validadores ('lib/validators/**'): reglas de formulario y mapeo de errores.
- Servicios ('lib/services/**'): ImageKit, Stripe, navegacion.

Mejoras posibles

- 'app/globals.css' esta muy grande y mezcla muchas responsabilidades; podria separarse por dominios (header, cards, modales, home, checkout, etc.).
- Existe carpeta 'components/shop/**' sin referencias activas detectadas, posible codigo legado no usado.

4. Dise o visual y sistema estetico

Estado actual (implementado)

La identidad visual es coherente con la estetica retro 80s/vaporwave + Windows 95/98:

- Paleta base: violetas, azules electricos, rosas neon sobre fondos oscuros (':root' y '@theme inline' en 'app/globals.css').
- Fondo general:
- Gradientes radiales + lineales en 'body'.
- Rejilla retro por seccion en '.page-background' con imagen 'rejilla.png'.
- Sistema de ventanas retro:
- '.win-window', '.win-tittlebar', '.win-button', '.win-input' (biselado, bordes duros, sin redondeo moderno).
- Sistema de cards:
- '.retro-concert-card' como base transversal para conciertos/noticias/media/store/account/carrito/checkout.
- Glow y neon:
- sombras 'drop-shadow', 'box-shadow' multicapa y pseudo-elementos con blur.
- Tipografia:
- Pixel ('Press Start 2P') para labels/titlebars.
- 'ByteBounce' en tabs de header.
- Sans retro para cuerpo y formularios.

Partes de codigo que generan el look

- Tokens + base: 'app/globals.css' (inicio del archivo).
- Header retro neon: bloque '.sb-header-*' (aprox lineas 989+).
- Ventanas y botones Win95: '.win-*' (aprox 1867+).
- Cards neon: '.retro-concert-card' y variantes (aprox 3457+).
- Home banda de videos + carrusel: '.home-videos-band-*', '.home-hero-carousel-*' (aprox 2590+ y 2843+).

Mejoras posibles

- Consolidar variantes repetidas de gradientes para reducir duplicacion.
- Definir guias de contraste mas estrictas en textos peque os de alto tracking.

5. Componentes principales

Header/Nav

- Ubicacion: 'components/layout/header-client.tsx'.
- Funcion:
- Renderiza icono busqueda, navegacion, logo central, perfil y carrito.
- Abre menu de perfil, modal auth, drawer carrito, buscador global.
- Estado:
- 'visibleCart', 'profileMenuOpen', 'authModalOpen', 'authMode', 'authRedirectTo', 'cartDrawerOpen'.
- Comunicacion:
- Evento 'AUTH_MODAL_OPEN_EVENT' para abrir login/register desde otros componentes.
- Evento 'CART_CLEARED_EVENT' para sincronizar badge al completar checkout.

Footer

- Ubicacion: 'components/layout/site-footer.tsx'.
- Funcion:
- Footer retro con logo, enlaces legales y sociales con glow.
- Estilo:
- Biselado Win95 + neon por icono social + microanimaciones de respiracion.

Hero/Home

- 'PageShell' ('components/ui/page-shell.tsx') monta cabecera de imagen + contenido.
- Home concreta:
- 'app/page.tsx' compone:
- banda de videos ('HomeVideosBand')
- carrusel de destacados ('HomeHeaderCarousel')

Cards

- Base reutilizable: '.retro-concert-card'.
- Usadas en:
- conciertos ('components/concerts/concert-cards-client.tsx')
- noticias ('components/band/news-list-client.tsx')
- fotos/videos/catalogos media ('app/media/*/page.tsx')
- store ('components/store/store-product-card.tsx')
- cuenta/carrito/checkout ('app/account/page.tsx', 'app/carrito/page.tsx', 'components/checkout/checkout-flow.tsx')

Modales

- Sistema generico: 'components/ui/app-modal.tsx'.
- Modales especificos:
- auth ('components/auth/auth-modal.tsx')
- drawer carrito ('components/cart/cart-drawer.tsx')
- buscador rapido ('components/layout/global-search.tsx')
- detalle tienda interceptado ('app/store/@modal/(.)[slug]/page.tsx')
- bio/musica (usan 'AppModal').

Formularios

- Login: 'components/auth/login-form.tsx' + 'loginAction'.
- Registro: 'components/auth/register-form.tsx' + 'registerAction'.
- Checkout: 'components/checkout/checkout-flow.tsx' + '/api/checkout'.
- Store add to cart: 'components/store/store-product-purchase-form.tsx' + 'addToCartAction'.

Secciones funcionales

- Conciertos: 'components/concerts/*'.
- Banda/noticias/bio: 'components/band/*'.
- Media fotos/videos: 'components/media/*'.
- Musica catalogo/modales: 'components/music/*'.
- Store/producto/modal: 'components/store/*'.
- Carrito/checkout/cuenta: 'app/carrito', 'app/checkout', 'app/account'.

6. Dinamismo e interaccion

Estado actual (implementado)

- Menus desplegables:
- Dropdown en tabs con children del header ('header-client.tsx' + 'sb-header-dropdown').
- Menu perfil con click fuera para cerrar.
- Modales:
- Apertura/cierre por estado local y eventos.
- Cierre por overlay y 'Escape'.
- Bloqueo de scroll de 'body'.
- Búsqueda global:
- Debounce 250ms.
- Resultados de paginas + productos.
- Navegacion por teclado (flechas/enter/escape).
- Cards interactivas:
- Hover/scale/glow.
- Algunas abren modales (conciertos, musica, bio).
- Formularios:
- Validacion cliente + servidor en registro y checkout.
- Errores por campo y mensajes de estado.
- Filtros:
- Formularios GET para conciertos, noticias, musica, media, tienda.
- Modal de filtros en vista compacta.
- Paginacion:
- Reutilizada por secciones con flechas y paginas visibles.
- Checkout:
- Guardado de borrador en 'localStorage'.
- Restriccion de metodo (PayPal UI-only).
- Redireccion a Stripe.

Mejoras posibles

- Unificar mas la gestion de focus trap entre modales (hay implementaciones parecidas en varios componentes).
- Aæadir estados skeleton locales en listados antes de respuesta (actualmente hay loading boundaries por ruta, no siempre granular por widget).

7. Animaciones, glow y efectos visuales

Estado actual (implementado)

- Transiciones CSS:
- hover/active/focus en tabs, botones, cards, iconos.
- Keyframes personalizadas en 'app/globals.css':
- 'sb-header-shell-neon', 'sb-header-shell-scan', 'sb-header-tab-sheen'
- 'sb-concert-neon-breathe', 'sb-concert-neon-orbit'
- 'sb-footer-sheen', 'sb-footer-icon-breathe'
- etc.
- Glow/neon:
- 'drop-shadow' y 'box-shadow' por capas.
- Overlays:
- modales con fondos oscuros o vaporwave ('retro-vapor-overlay', 'retro-modal-overlay', 'global-search-overlay').
- Carruseles/bandas:
- progreso visual en home ('home-hero-carousel-progress-fill', 'home-videos-band-progress').

Accesibilidad de movimiento

- Hay bloque '@media (prefers-reduced-motion: reduce)' que desactiva varias animaciones.

Mejora importante detectada

- En 'app/globals.css' alrededor de '@keyframes sb-header-logo-float' (lineas ~5285+), hay una estructura aparentemente incorrecta: se incluyen selectores normales dentro del bloque de keyframes. Esto puede romper parseo CSS en cascada y generar comportamiento impredecible.

8. Responsive design

Estado actual (implementado)

- Se usan breakpoints por clases Tailwind y media queries CSS:
- 'max-width: 639px', '640px', '767px', '768px', '900px', '1024px', '1280px', '1560px'.
- Ajustes por seccion:
- Grids de cards cambian columnas en mobile/tablet/desktop.
- Modales limitan alto y usan 'overflow-y-auto'.
- Home ajusta tamaæo de tarjetas/videos por breakpoints.
- Filtros pasan de panel lateral a modal icono en pantallas no grandes.

Hallazgos relevantes

- El header desktop ('.sb-header-desktop-nav') se oculta en 'max-width: 1560px'

('app/globals.css'), y no se detecta en 'header-client.tsx' un menu hamburguesa funcional que sustituya la navegacion principal. Esto puede dejar navegacion principal menos visible en pantallas laptop habituales.

Mejoras posibles

- Definir un nav movil explicito (drawer/menu) para el rango <1560.
- Revisar consistencia de alturas minimas de cards para evitar huecos visuales en datasets cortos.

9. Gestion de estado

Estado actual (implementado)

- Estado local con 'useState/useEffect/useMemo/useRef' en componentes cliente.
- No hay estado global con Context/Redux/Zustand.
- Eventos de ventana para sincronizacion puntual:
- 'sugarbay:auth-modal-open'
- 'sugarbay:cart-cleared'
- Formularios server-driven:
- 'useActionState' + Server Actions para login/registro/add-to-cart.
- Estado persistente local:
- Checkout draft en 'localStorage'.
- Sync puntual en 'sessionStorage' para evitar clear duplicado post-checkout.

Auth y cart

- Auth session se resuelve en servidor ('lib/auth/dal.ts').
- Cart se hidrata desde servidor en header ('components/layout/site-header.tsx'), luego cliente lo gestiona visualmente.

Mejoras posibles

- Un pequeño estado global de UI (solo modales y notificaciones) podria reducir eventos custom y logica repetida.

10. Routing y navegacion

Estado actual (implementado)

- App Router con 24 paginas ('app/**/page.tsx').
- Redirect roots:
- '/concerts' -> '/concerts/upcoming'
- '/band' -> '/band/news'
- '/media' -> '/media/photos'
- Rutas dinamicas:
- 'app/store/[slug]/page.tsx'
- 'app/media/photos/[slug]/page.tsx'
- 'app/media/videos/[slug]/page.tsx'
- Modal routing avanzado:
- 'app/store/@modal/(.)[slug]/page.tsx' (intercepting route) para abrir detalle de

producto como modal sobre listado.

Diferencia navegacion real vs modal

- Navegacion real: 'Link' a ruta nueva.
- Modal en store: misma entidad puede abrirse como ruta full ('/store/[slug]') o como modal interceptado sobre '/store'.

Mejoras posibles

- Gestionar estado activo en tabs del header (hay clase CSS '.sb-header-tab-active' pero no se aplica en JSX actual).

11. Imagenes y recursos

Estado actual (implementado)

- 'next/image' muy extendido.
- URL remotas de ImageKit y thumbnails YouTube.
- 'resolveImageUrl()' ('lib/services/imagekit.ts') normaliza rutas relativas/absolutas.
- Control de encuadre:
- 'object-cover', 'object-contain', 'object-top', 'object-center' segun caso.
- Home header usa cabeceras grandes en 'PageShell' con 'mask-image' para fade inferior.

Rendimiento de imagen

- Algunas imagenes clave usan 'priority'.
- En 'PageShell', imagen de cabecera se declara con 'unoptimized' (reduce trabajo de optimizacion de Next pero puede afectar transferencia/CDN segun origen).

Mejoras posibles

- Revisar 'unoptimized' en cabeceras para decidir si mantenerlo o volver a pipeline de 'next/image' optimizado.
- A adir 'placeholder'/blur en imagenes grandes para mejor percepcion de carga.

12. Formularios y validaciones

Estado actual (implementado)

- Login:
- 'components/auth/login-form.tsx'
- valida schema en servidor ('loginSchema', 'lib/validators/auth.ts').
- Registro:
- 'components/auth/register-form.tsx'
- validacion cliente previa con 'registerSchema' + validacion server final.
- reglas avanzadas: edad minima, password fuerte, username anti-similitud.
- Checkout:
- 'components/checkout/checkout-flow.tsx'
- validacion en cliente (schema) + backend en '/api/checkout'.
- mapeo de errores por campo con 'mapCheckoutIssuesToFieldErrors'.

- Carrito:
- formularios server action para actualizar cantidad/eliminar.

Relacion con backend

- Front recoge datos y valida UX.
- Servidor revalida siempre antes de persistir/crear pagos.

Mejoras posibles

- Unificar estilo visual de errores en todos los formularios (algunos siguen paletas legacy 'sb-*').
- Añadir throttling/anti-abuso visible para auth si se requiere endurecimiento.

13. Accesibilidad

Estado actual (implementado)

- Uso correcto de roles/dialog en modales.
- Botones de cierre con 'aria-label'.
- 'aria-expanded', 'aria-controls', 'aria-current', 'aria-live' en varios flujos.
- Focus visible global con ':focus-visible'.
- Trap de foco en modales principales.
- Atajos teclado:
- 'Ctrl/Cmd + K' para buscador.
- Etiquetado de inputs con 'label htmlFor'.

Gaps/mejoras posibles

- Revisar contraste de ciertos textos muy pequeños en fondos complejos.
- Confirmar coherencia de heading hierarchy visual vs semantic en algunas cards que usan 'h2/h3' según contexto.
- Asegurar menú de navegación alternativo en mobile/laptop si desktop nav se oculta.

14. Rendimiento

Estado actual (implementado)

- Uso fuerte de Server Components para páginas de catálogo y carga inicial.
- Client Components solo donde hay interacción.
- 'Promise.all' en varias páginas para paralelizar lecturas ('app/page.tsx', 'app/checkout/page.tsx').
- Route-level loading y error boundaries:
- '17' loading files.
- '10' error boundaries.
- Debounce en buscador y 'AbortController' para cancelar requests previas.

Riesgos observados

- 'app/globals.css' muy grande (coste de mantenimiento y posibles conflictos).
- Varias animaciones activas simultáneas pueden penalizar equipos modestos.

- Header nav oculto <1560 puede empujar a mas clics/context switches para navegar.

Mejoras posibles

- Reducir peso CSS global por modulos tematicos.
- Introducir lazy-load granular de widgets pesados donde aplique.
- Auditar Web Vitals (LCP/CLS/INP) y optimizar cabeceras visuales grandes.

15. Seguridad front-end

Estado actual (implementado)

- Proteccion de rutas privadas en 'proxy.ts' ('/account', '/carrito', '/checkout').
- Variables sensibles gestionadas en servidor ('lib/env.ts' + 'requireEnv' para Stripe secret).
- Validacion cliente + servidor (no se confia solo en cliente).
- Checkout en backend con Stripe session server-side.
- Webhook Stripe para confirmar estado real de pago.

Riesgos controlados y a vigilar

- URLs externas (Mapas, embeds, links externos): se usa 'target="_blank"' y 'rel="noreferrer"' en muchos casos, correcto.
- El cliente nunca debe exponer secretos (actualmente no se observa uso de secretos en cliente, correcto).
- Acciones de carrito y auth dependen de session en servidor (correcto).

Mejoras posibles

- Considerar cabeceras CSP mas estrictas para embeds.
- A adir auditoria automatica de dependencias y SAST en CI.

16. Problemas encontrados o posibles puntos debiles

1. Posible error estructural en CSS:

- 'app/globals.css' en '@keyframes sb-header-logo-float' contiene selectores de clase dentro del keyframe (lineas ~5285+), probable bug de parseo.

2. Navegacion principal en header:

- '.sb-header-desktop-nav' se oculta en 'max-width: 1560px' sin evidencia de menu alternativo funcional en JSX.

3. Clases CSS aparentemente no utilizadas:

- '.sb-header-menu-btn', '.sb-header-mobile-item', '.sb-header-brand-mobile'.

4. Clase activa de tabs no aplicada:

- existe '.sb-header-tab-active', pero no se aplica en 'header-client.tsx'.

5.Codigo potencialmente legado:

- carpeta 'components/shop/*' sin imports detectados.

6. Inconsistencias de codificacion de texto:

- algunos literales se muestran con caracteres extra os en salida de terminal (posible encoding/cp), conviene revisar consistencia UTF-8.

7. Limitacion de verificaciones en este analisis:

- en este entorno no fue posible ejecutar 'npm'/'node' para lint/build/test automaticos.

17. Mejoras futuras

Prioridad alta

1. Corregir bloque CSS malformado en 'app/globals.css' (keyframes/logo).
2. Implementar navegacion responsive real para header en anchos menores.
3. Activar estado visual de ruta activa en tabs del header.

Prioridad media

1. Refactor de CSS global por modulos (header/cards/modales/home/checkout).
2. Limpieza de codigo no usado ('components/shop', clases huérfanas).
3. Homogeneizar textos y encoding UTF-8.

Prioridad evolutiva

1. Testing visual/regresion (Playwright snapshots).
2. Instrumentacion Web Vitals y presupuesto de performance.
3. Mejoras de a11y avanzadas (auditoria automatica + contraste sistematico).

18. Explicacion para defensa oral

Version de 1 minuto

"El front-end de Sugarbay esta hecho con Next.js App Router, React 19 y TypeScript estricto. Usamos Server Components para cargar catalogos y datos desde Prisma de forma eficiente, y Client Components para interacciones como modales, buscador global, carrito y checkout. A nivel visual, construimos un sistema propio retro-vaporwave con Tailwind y CSS global: ventanas estilo Windows 95, bordes biselados, glow neon y layouts responsive. El resultado es una web de contenido musical y e-commerce con identidad fuerte, rutas dinamicas, filtros, paginacion, autenticacion y pago seguro con Stripe."

Version tecnica de 3 minutos

"La app se organiza en 'app/', 'components/' y 'lib/'. En 'app/' definimos rutas y carga de datos server-side; en 'components/' tenemos UI por dominio (concerts, band, media, music, store, auth, cart, checkout); y en 'lib/' va la logica de repositorios, validacion y servicios externos.

El shell global esta en 'app/layout.tsx' con 'SiteHeader', 'PageShell' y 'SiteFooter'. El header cliente gestiona estados de perfil, auth modal y carrito con 'useState', eventos custom y cierres por click fuera. El buscador rapido usa debounce + '/api/search', soporta teclado y combina paginas y productos.

Visualmente hay dos sistemas: 'sb-*' para superficies neon y 'win-*' para ventanas retro. Las cards reutilizan '.retro-concert-card' en conciertos, noticias, media, store, cuenta y checkout. Los modales usan 'AppModal' o variantes especializadas con trap de foco, 'Escape', overlay y scroll lock.

En datos, las paginas usan repositorios Prisma y 'Promise.all'. En formularios, registro y checkout validan con Zod en cliente y servidor. El carrito usa Server Actions para anadir/actualizar/eliminar y revalidacion de rutas. El checkout crea sesion Stripe en backend, y webhook actualiza estado de orden. Ademas, rutas privadas ('/account', '/carrito', '/checkout') se protegen desde 'proxy.ts'.

Como mejora detectada, hay una anomalia en 'globals.css' en un bloque de keyframes y un gap responsive en navegacion del header bajo 1560px."

Version completa de 5 minutos

"Arquitectonicamente, el proyecto aprovecha bien App Router: pages server para datos y componentes client solo en puntos interactivos. Esto reduce JS de cliente en listados y mantiene UX dinamica donde importa.

El home ('app/page.tsx') compone dos piezas: banda de videos auto-scroll con pausa en hover/focus y carrusel de destacados de concierto/tienda/noticia. Ambas consumen datos reales de repositorio, no mocks visuales.

En contenido, cada seccion tiene filtros/paginacion con un patron uniforme: formulario GET + serializacion de query params + UI retro en modal de filtros para compacto. Ese patron se repite en conciertos, noticias, musica, fotos, videos y store, lo que mejora consistencia.

El store tiene una decision avanzada de UX/routing: detalle de producto como pagina normal y como modal interceptado ('@modal/(.)[slug]'), permitiendo explorar sin perder contexto de catalogo. El add-to-cart usa Server Action con estado 'auth_required' para disparar login modal sin romper flujo.

Auth y checkout estan bien acoplados a seguridad server-side: validacion Zod en ambos lados, sesion por cookie, proteccion de rutas, creacion de pago en backend y confirmacion por webhook.

A nivel estetico, el sistema visual esta muy trabajado: tipografias pixel + display custom, titlebars retro, biseles, sombras duras y neon. Hay buena base de accesibilidad: labels, roles, 'aria-*', focus visible, cierre de modales por teclado y soporte para reduced-motion.

En rendimiento, hay buenas practicas como server rendering, 'Promise.all', cancellation de fetch y boundaries de loading/error por ruta. Como deuda tecnica principal: 'globals.css' es muy grande y tiene una inconsistencia estructural puntual en keyframes; tambien falta una estrategia clara de nav responsive bajo 1560px y conviene limpiar codigo/clases legacy.

En resumen, el front-end combina identidad visual fuerte, modularidad por dominio y flujos reales de e-commerce, con una base robusta y mejoras claras para una siguiente iteracion."

19. Preguntas posibles del tribunal y respuestas

1. ¿Por que usaste Next.js App Router?

- Porque permite SSR/RSC de forma nativa, mejor SEO y mejor control de carga de datos por ruta, manteniendo interactividad solo donde hace falta.

2. ¿Donde se organiza el front-end?

- En 'app/' (rutas/layout), 'components/' (UI por dominio), 'lib/' (repositorios, validaciones, servicios).

3. ¿Como funcionan los componentes principales?

- El shell global monta header/footer/page-shell. Cada seccion compone cards, filtros y paginacion. Interacciones avanzadas se encapsulan en cliente (modales, buscador, carrito, formularios).

4. ¿Donde se gestiona el estado?

- Mayormente local ('useState/useEffect/useMemo') por componente. No hay store global; se usan eventos custom para sincronizaciones puntuales (auth/cart).

5. ¿Como se abren los modales?

- Por estado local o por evento global ('AUTH_MODAL_OPEN_EVENT'). Se cierran con overlay, boton close y 'Escape'.

6. ¿Como aplicas glow y estilo retro?

- Con clases 'win-*', 'retro-*', 'sb-*' en 'app/globals.css': biseles, titlebars, sombras multicapa, pseudo-elementos y keyframes custom.

7. ¿Como se consigue responsive?

- Tailwind + media queries personalizadas. Se ajustan grids, tamaños, paddings, modales y widgets por breakpoints.

8. ¿Como cargas imagenes?

- Con 'next/image' y helper 'resolveImageUrl' para ImageKit/URLs externas; en cabeceras grandes se usa 'PageShell' con control de 'object-fit' y mascara inferior.

9. ¿Que partes son dinamicas?

- Búsqueda en vivo, dropdowns, modales, carruseles, filtros, paginacion, formularios auth/checkout, carrito y flujo Stripe.

10. ¿Como conecta front con DB/API?

- Paginas server llaman repositorios Prisma. En cliente, acciones/formularios consumen Server Actions o route handlers ('/api/search', '/api/checkout').

11. ¿Diferencia entre server y client components en tu proyecto?

- Server para render inicial y datos; client para eventos, estado local y navegacion interactiva.

12. ¿Que mejorarias si tuvieras mas tiempo?

- Corregir keyframes CSS malformado, reforzar nav responsive, modularizar CSS global, limpiar legado y añadir tests visuales.

13. ¿Problemas reales encontrados?

- Complejidad de estilos globales por el alto nivel de personalizacion, y necesidad de mantener coherencia responsive/visual en muchas secciones.

14. ¿Por que no usaste estado global tipo Redux?

- El dominio actual se resuelve bien con estado local + server rendering + eventos puntuales; mantenerlo simple reduce sobrecarga.

20. Conclusion final

El front-end de Sugarbay esta tecnicamente bien planteado para un proyecto mixto de branding musical + e-commerce:

- base moderna (Next 16 + React 19 + TS strict),

- arquitectura modular por dominio,
- fuerte identidad visual retro/vaporwave coherente,
- interaccion rica (modales, filtros, carruseles, buscador, checkout),
- y conexion real con backend/Stripe.

No es un prototipo superficial: hay decisiones de producto y arquitectura avanzadas (modal interceptado en store, validacion dual, sincronizacion de carrito post-pago, patrones reutilizables de cards/filtros/paginacion).

Los principales puntos de mejora estan localizados y abordables: limpieza y modularizacion de CSS global, ajuste de navegacion responsive del header y correccion de una inconsistencia puntual en keyframes. Con esas mejoras, la base quedaria aun mas solida para evolucion productiva y escalado.

Comprobaciones realizadas para este informe

- Lectura directa de codigo real (no inferencias sin fuente de repo).
- Revision de rutas, componentes, estilos globales, validaciones, actions y handlers.
- Conteos utiles:
 - '24' paginas 'app/**/page.tsx'
 - '17' 'loading.tsx'
 - '10' 'error.tsx'
 - '4' route handlers API
 - '39' componentes/archivos con "'use client'"
- Limitacion del entorno:
 - no fue posible ejecutar 'npm/node' para lint/build/test en esta sesion.